

▼ InfoNCE Loss + CLIPModel Integration

▼ Setup

```
from google.colab import drive
drive.mount('/content/drive')

!git clone https://github.com/prajwl7676/mini-clip-DLAI.git

%cd /content/mini-clip-DLAI
!pip install -q -r requirements.txt
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
fatal: destination path 'mini-clip-DLAI' already exists and is not an empty directory.
/content/mini-clip-DLAI

```

_____ 44.4/44.4 kB 1.5 MB/s eta 0:00:00
_____ 61.0/61.0 kB 2.4 MB/s eta 0:00:00
_____ 57.6/57.6 kB 3.7 MB/s eta 0:00:00
_____ 906.4/906.4 MB 601.9 kB/s eta 0:00:00
_____ 7.2/7.2 MB 84.3 MB/s eta 0:00:00
_____ 9.7/9.7 MB 55.4 MB/s eta 0:00:00
_____ 480.6/480.6 kB 23.8 MB/s eta 0:00:00
_____ 4.5/4.5 MB 80.7 MB/s eta 0:00:00
_____ 18.0/18.0 MB 73.7 MB/s eta 0:00:00
_____ 8.3/8.3 MB 86.3 MB/s eta 0:00:00
_____ 78.3/78.3 kB 5.7 MB/s eta 0:00:00
_____ 12.9/12.9 MB 86.7 MB/s eta 0:00:00
_____ 363.4/363.4 MB 4.7 MB/s eta 0:00:00
_____ 13.8/13.8 MB 107.9 MB/s eta 0:00:00
_____ 24.6/24.6 MB 79.9 MB/s eta 0:00:00
_____ 883.7/883.7 kB 56.7 MB/s eta 0:00:00
_____ 664.8/664.8 MB 2.2 MB/s eta 0:00:00
_____ 211.5/211.5 MB 6.2 MB/s eta 0:00:00
_____ 56.3/56.3 MB 15.5 MB/s eta 0:00:00
_____ 127.9/127.9 MB 8.8 MB/s eta 0:00:00
_____ 207.5/207.5 MB 3.7 MB/s eta 0:00:00
_____ 188.7/188.7 MB 6.8 MB/s eta 0:00:00
_____ 21.1/21.1 MB 44.8 MB/s eta 0:00:00
_____ 99.1/99.1 kB 11.2 MB/s eta 0:00:00
_____ 6.2/6.2 MB 75.0 MB/s eta 0:00:00
_____ 209.6/209.6 MB 4.1 MB/s eta 0:00:00
_____ 179.3/179.3 kB 19.5 MB/s eta 0:00:00
_____ 566.4/566.4 kB 50.3 MB/s eta 0:00:00
_____ 3.1/3.1 MB 117.4 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is temporary and will be removed in a future version of pip. It is recommended to keep pip's dependency resolver up to date by upgrading it to the latest version. See <https://pip.pypa.io/en/stable/news/2021-06-07-deprecating-old-dependency-resolver/> for more details.

xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
pytorch 2.3.8 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.9.0 which is incompatible.
jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
cupy-cuda12x 14.0.1 requires numpy<2.6, >=2.0, but you have numpy 1.26.4 which is incompatible.
opencv-python-headless 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
hdbscan 0.8.43 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
shap 0.51.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
opencv-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
dataproc-spark-connect 1.1.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.
rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
tiffio 2026.4.11 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
tobler 0.14.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
tobler 0.14.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.
opencv-contrib-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.

```
import sys
sys.path.insert(0, '/content/mini-clip-DLAI')
!pwd
!ls -l
```

```
/content
total 12
drwx----- 5 root root 4096 Jun  5 11:15 drive
```

```
drwxr-xr-x 6 root root 4096 Jun  5 11:15 mini-clip-DLAI
drwxr-xr-x 1 root root 4096 May 26 13:31 sample_data
```

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
import numpy as np
from datasets import load_dataset
from transformers import DistilBertTokenizer
from torch.utils.data import DataLoader

from src.model_architecture import CLIPModel
from src.dataset import FlickrDataset, get_transform

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')
```

```
Device: cuda
```

✓ Implement InfoNCE loss

```
def clip_loss(image_embeddings, text_embeddings, temperature):

    B = image_embeddings.shape[0]

    # Step 1 - N×N cosine similarity matrix
    # S[i,j] = dot product of image_i and text_j (cosine sim since L2-normed)
    S = image_embeddings @ text_embeddings.T          # (B, B)

    # Step 2 - scale by temperature
    logits = S / temperature                        # (B, B)

    # Step 3 - labels: image i is matched with text i
    labels = torch.arange(B, device=image_embeddings.device)

    # Step 4 - symmetric cross-entropy
    # Row i: which column j maximises logits[i,j]? Answer should be j=i
    # Col j: which row i maximises logits[i,j]? Answer should be i=j
    loss_i2t = F.cross_entropy(logits, labels)        # image → text
    loss_t2i = F.cross_entropy(logits.T, labels)      # text → image

    return (loss_i2t + loss_t2i) / 2.0

print('clip_loss defined.')
```

clip_loss defined.

✓ Sanity check: what does the loss value mean?

For a batch of N pairs, a random model produces loss $\approx \log(N)$.

This is the baseline — the model is guessing randomly.

After training, the loss should be well below $\log(N)$.

```
import math

for B in [8, 32, 64, 128]:
    expected_random_loss = math.log(B)
    print(f'Batch size {B:4d} → random loss baseline  $\approx \log(\{B\}) = \{expected\_random\_loss:.3f\}$ ')

print()
print('If your trained model achieves loss  $\ll \log(B)$ , it has learned something.')
print('If loss  $\approx \log(B)$ , the model is still random — something is wrong.')
```

```
Batch size    8 → random loss baseline  $\approx \log(8) = 2.079$ 
Batch size   32 → random loss baseline  $\approx \log(32) = 3.466$ 
Batch size   64 → random loss baseline  $\approx \log(64) = 4.159$ 
Batch size  128 → random loss baseline  $\approx \log(128) = 4.852$ 
```

If your trained model achieves loss $\ll \log(B)$, it has learned something.
 If loss $\approx \log(B)$, the model is still random – something is wrong.

✓ Test loss on fake embeddings

Three scenarios:

1. **Random embeddings** → loss should be $\approx \log(B)$
2. **Perfect embeddings** (diagonal = 1, off-diagonal = 0) → loss should be ≈ 0
3. **Swapped embeddings** (diagonal = 0) → loss should be very high

```
B, D = 8, 256
tau = torch.tensor(0.07)

# Scenario 1: random embeddings
img_rand = F.normalize(torch.randn(B, D), dim=-1)
txt_rand = F.normalize(torch.randn(B, D), dim=-1)
loss_random = clip_loss(img_rand, txt_rand, tau)

# Scenario 2: perfect embeddings – image i == text i
perfect = F.normalize(torch.randn(B, D), dim=-1)
loss_perfect = clip_loss(perfect, perfect.clone(), tau)

# Scenario 3: worst case – image i matched with text (i+1)
txt_wrong = torch.roll(perfect, shifts=1, dims=0) # shift captions by 1
loss_worst = clip_loss(perfect, txt_wrong, tau)

print(f'Random embeddings loss : {loss_random.item():.4f} (expected  $\approx \{\mathbf{math.log(B):.3f}\}$ )')
print(f'Perfect match loss      : {loss_perfect.item():.4f} (expected  $\approx 0.0$ )')
print(f'Worst case loss         : {loss_worst.item():.4f} (expected  $\gg \{\mathbf{math.log(B):.3f}\}$ )')

assert loss_perfect.item() < 0.01, 'Perfect loss should be near 0'
assert loss_random.item() > 1.0, 'Random loss should be > 1'
assert loss_worst.item() > loss_random.item(), 'Worst case > random'
print('\nAll loss sanity checks passed.')

Random embeddings loss : 2.2550 (expected  $\approx 2.079$ )
Perfect match loss      : 0.0000 (expected  $\approx 0.0$ )
Worst case loss         : 14.1017 (expected  $\gg 2.079$ )

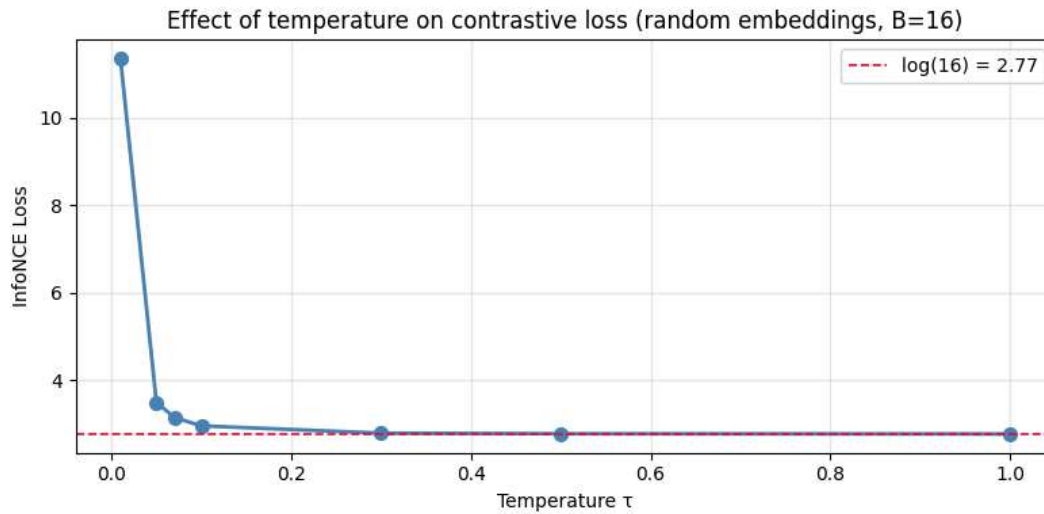
All loss sanity checks passed.
```

✓ Effect of temperature on loss

```
img = F.normalize(torch.randn(16, 256), dim=-1)
txt = F.normalize(torch.randn(16, 256), dim=-1)

temps = [0.01, 0.05, 0.07, 0.1, 0.3, 0.5, 1.0]
losses = [clip_loss(img, txt, torch.tensor(t)).item() for t in temps]

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(temps, losses, 'o-', color='steelblue', linewidth=2, markersize=7)
ax.axhline(math.log(16), color='crimson', linestyle='--', linewidth=1.2, label=f'log(16) = {math.log(16):.2f}')
ax.set_xlabel('Temperature  $\tau$ ')
ax.set_ylabel('InfoNCE Loss')
ax.set_title('Effect of temperature on contrastive loss (random embeddings, B=16)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('assets/temperature_effect.png', dpi=120, bbox_inches='tight')
plt.show()
print('Saved to assets/temperature_effect.png')
```



Saved to assets/temperature_effect.png

✓ Load real data for a complete integration test

```
dataset_hf = load_dataset('nlphuji/flickr30k', split='test[:500]')
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

ds = FlickrDataset(dataset_hf, tokenizer, get_transform(train=True))
loader = DataLoader(ds, batch_size=32, shuffle=True, num_workers=2,
                    pin_memory=True, drop_last=True)

images, tokens = next(iter(loader))
images = images.to(device)
input_ids = tokens['input_ids'].to(device)
attn_mask = tokens['attention_mask'].to(device)

print(f'Batch ready - images: {images.shape}, tokens: {input_ids.shape}')

Batch ready - images: torch.Size([32, 3, 224, 224]), tokens: torch.Size([32, 77])
```

✓ Full integration: model → loss → backward

```

model = CLIPModel(embedding_dim=256).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)

# One training step
model.train()

# Step 1: forward pass
img_emb, txt_emb, tau = model(images, input_ids, attn_mask)

# Step 2: compute loss
loss = clip_loss(img_emb, txt_emb, tau)

# Step 3: backward pass
optimizer.zero_grad()
loss.backward()

# Step 4: gradient clipping (prevents exploding gradients)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# Step 5: update weights
optimizer.step()

print(f'loss after step 1 : {loss.item():.4f}')
print(f'temperature  $\tau$  : {tau.item():.4f}')
print(f'log(32) baseline : {math.log(32):.4f}')
print()
print('One complete training step succeeded.')

```

Downloading: "<https://download.pytorch.org/models/resnet50-0676ba61.pth>" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth [100%|██████████| 97.8M/97.8M [00:00<00:00, 131MB/s]

model.safetensors: 100% 268M/268M [00:02<00:00, 173MB/s]

loss after step 1 : 3.5187
temperature τ : 0.0700
log(32) baseline : 3.4657

One complete training step succeeded.

✓ Mini training loop: 10 steps to confirm loss decreases

```

model = CLIPModel(embedding_dim=256).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)

loss_history = []

# Run 10 steps on the same batch (overfit test)
# Loss should decrease – proves the gradients are flowing correctly
for step in range(10):
    model.train()
    img_emb, txt_emb, tau = model(images, input_ids, attn_mask)
    loss = clip_loss(img_emb, txt_emb, tau)

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()

    loss_history.append(loss.item())
    print(f'Step {step+1:2d} | loss: {loss.item():.4f} |  $\tau$ : {tau.item():.4f}')

print()
if loss_history[-1] < loss_history[0]:
    print('Loss is decreasing – gradients are flowing correctly.')
else:
    print('WARNING: loss did not decrease – check model and loss function.')

```

```

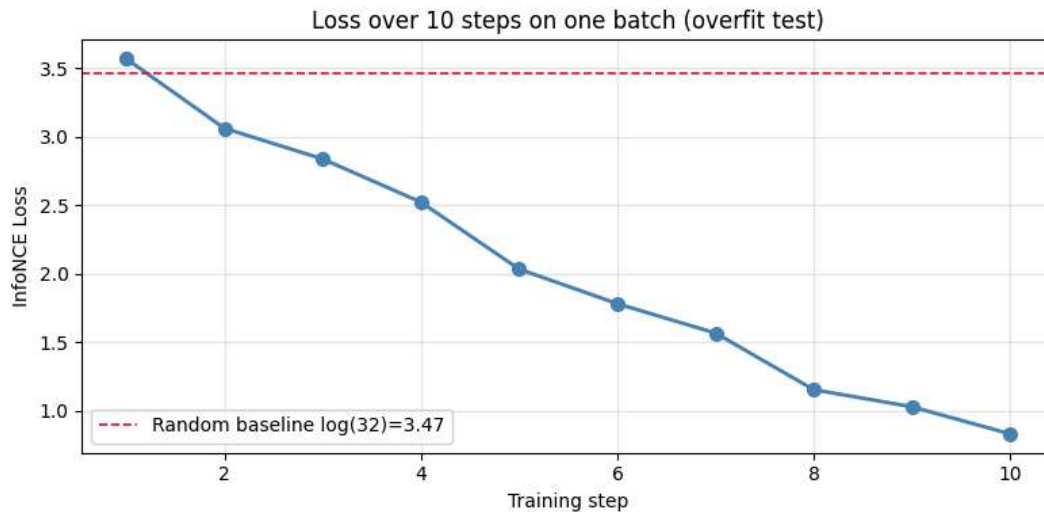
Step 1 | loss: 3.5716 |  $\tau$ : 0.0700
Step 2 | loss: 3.0626 |  $\tau$ : 0.0700
Step 3 | loss: 2.8385 |  $\tau$ : 0.0700
Step 4 | loss: 2.5232 |  $\tau$ : 0.0700
Step 5 | loss: 2.0335 |  $\tau$ : 0.0700
Step 6 | loss: 1.7808 |  $\tau$ : 0.0700
Step 7 | loss: 1.5656 |  $\tau$ : 0.0700
Step 8 | loss: 1.1521 |  $\tau$ : 0.0700

```

```
Step 9 | loss: 1.0271 |  $\tau$ : 0.0700
Step 10 | loss: 0.8286 |  $\tau$ : 0.0700
```

Loss is decreasing – gradients are flowing correctly.

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(1, 11), loss_history, 'o-', color='steelblue', linewidth=2, markersize=7)
ax.axhline(math.log(32), color='crimson', linestyle='--', linewidth=1.2,
            label=f'Random baseline log(32)={math.log(32):.2f}')
ax.set_xlabel('Training step')
ax.set_ylabel('InfoNCE Loss')
ax.set_title('Loss over 10 steps on one batch (overfit test)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('assets/overfit_test_loss.png', dpi=120, bbox_inches='tight')
plt.show()
```



Visualise similarity matrix AFTER 10 steps

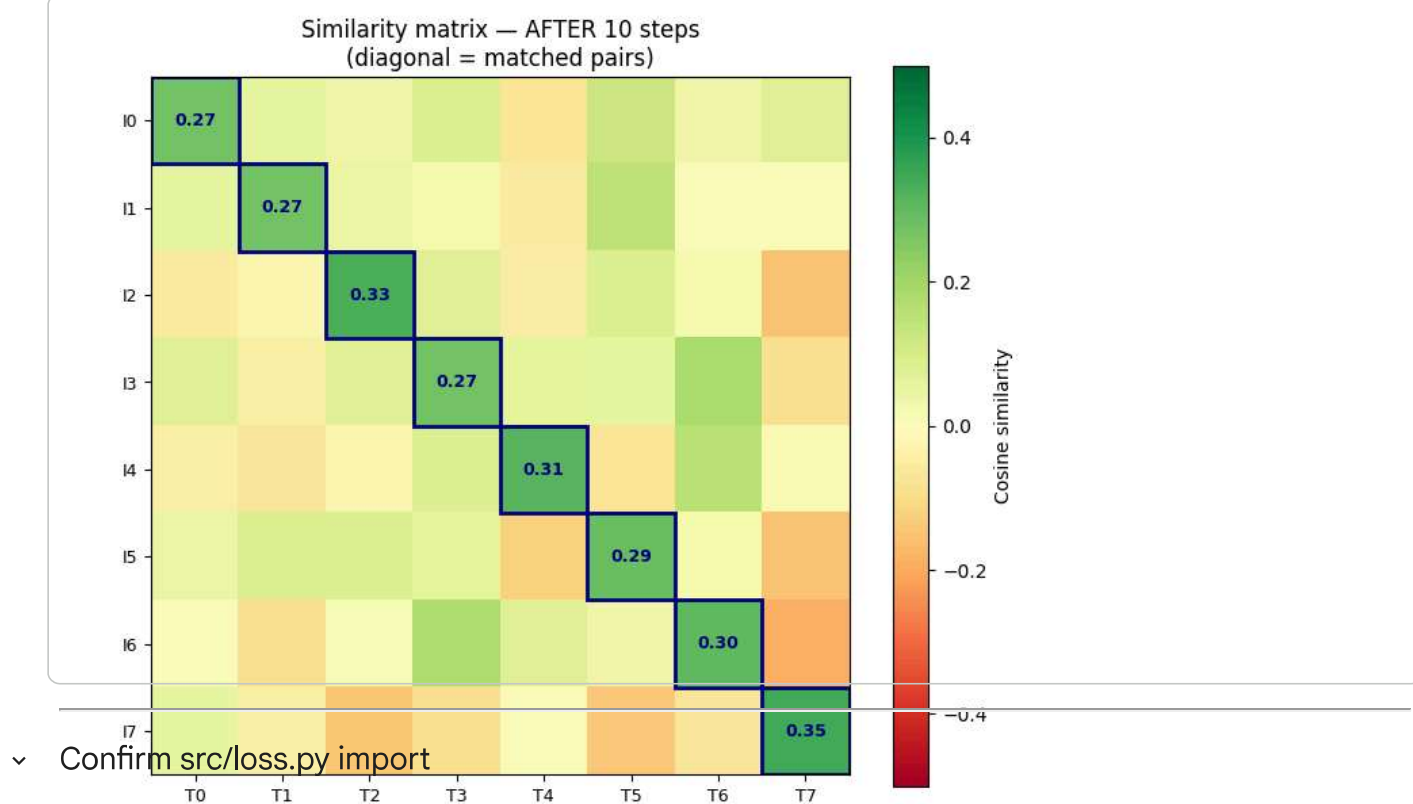
The diagonal should already be slightly brighter after just 10 steps.

```
model.eval()
with torch.no_grad():
    img_emb, txt_emb, tau = model(images[:8], input_ids[:8], attn_mask[:8])
    sim = (img_emb @ txt_emb.T).cpu().numpy()

fig, ax = plt.subplots(figsize=(7, 6))
im = ax.imshow(sim, cmap='RdYlGn', vmin=-0.5, vmax=0.5)
ax.set_xticks(range(8))
ax.set_yticks(range(8))
ax.set_xticklabels([f'T{i}' for i in range(8)], fontsize=9)
ax.set_yticklabels([f'I{i}' for i in range(8)], fontsize=9)
ax.set_title('Similarity matrix – AFTER 10 steps\n(diagonal = matched pairs)')

for i in range(8):
    ax.add_patch(plt.Rectangle((i-0.5, i-0.5), 1, 1, fill=False,
                              edgecolor='navy', lw=2))
    ax.text(i, i, f'{sim[i,i]:.2f}', ha='center', va='center',
            fontsize=9, fontweight='bold', color='navy')

plt.colorbar(im, ax=ax, label='Cosine similarity')
plt.tight_layout()
plt.savefig('assets/similarity_matrix_after_10_steps.png', dpi=120, bbox_inches='tight')
plt.show()
```



```
from src.loss import clip_loss as clip_loss_src, similarity_matrix
```

```
# Test imported loss matches inline version
img_t = F.normalize(torch.randn(8, 256), dim=-1)
txt_t = F.normalize(torch.randn(8, 256), dim=-1)
```